Building an Airflow ETL Pipeline

Currency Exchange Rate ETL with Apache Airflow (Astro CLI) and MongoDB

1. Objective

Build an ETL pipeline using the Astro CLI (Apache Airflow) that fetches daily currency exchange rates from the Frankfurter API, transforms them into a normalized schema, and loads them into MongoDB. The pipeline consists of exactly three tasks — Extract, Transform, and Load — which pass data between each other exclusively through named Airflow XComs.

2. Architecture Overview

Data source: Frankfurter API (https://api.frankfurter.dev/v1/latest), base currency EUR, targets USD/GBP/JPY.

Orchestration: Apache Airflow, run locally via the Astro CLI (astro dev start).

Storage: MongoDB 7.0, with Mongo Express as a web-based admin UI.

Both MongoDB and Mongo Express run as additional Docker services alongside Airflow's own containers, sharing the same Docker network so the DAG can reach MongoDB by its service hostname (mongodb). MongoDB data persists in a named Docker volume (mongodb_data), surviving container restarts.

3. Pipeline Tasks

3.1 Extract (extract_rates)

Calls the Frankfurter API for EUR -> USD, GBP, JPY and pushes the raw JSON response via a named XCom key ("raw_rates").

3.2 Transform (transform_rates)

Pulls the raw payload from the named XCom, and reshapes each currency quote into one document per target currency, matching the required schema:

```
{
  "base_currency": "EUR",
  "target_currency": "USD",
  "exchange_rate": 1.1165,
  "rate_date": "2026-11-06",
  "source": "Frankfurter API",
  "retrieved_at": "2026-11-07T15:55:33.656262"
}
```

The transformed list of records is pushed back out via a second named XCom key

("transformed_rates").

3.3 Load (load_to_mongo)

Pulls the transformed records from the named XCom and inserts them into MongoDB (database currency_db, collection exchange_rates) using pymongo.

4. DAG Source Code

File: dags/currency_etl_dag.py

```
"""
ETL pipeline: fetch daily EUR exchange rates from the Frankfurter API,
transform into a normalized schema, and load into MongoDB.

Tasks communicate exclusively through named XComs:
  extract_rates   -> pushes "raw_rates" (key="raw_rates")
  transform_rates -> pulls "raw_rates", pushes "transformed_rates"
  load_to_mongo   -> pulls "transformed_rates"
"""

from __future__ import annotations
from datetime import datetime
import requests
from airflow.sdk import dag, task

BASE_CURRENCY = "EUR"
TARGET_CURRENCIES = ["USD", "GBP", "JPY"]
FRANKFURTER_URL = "https://api.frankfurter.dev/v1/latest"

MONGO_CONN_URI = "mongodb://mongo_user:mongo_password@mongodb:27017/"
MONGO_DB_NAME = "currency_db"
MONGO_COLLECTION_NAME = "exchange_rates"

RAW_RATES_XCOM_KEY = "raw_rates"
TRANSFORMED_RATES_XCOM_KEY = "transformed_rates"


@dag(
    dag_id="currency_exchange_etl",
    schedule="@daily",
    start_date=datetime(2024, 1, 1),
    catchup=False,
    tags=["etl", "currency", "mongodb"],
)
def currency_exchange_etl():

    @task(task_id="extract_rates")
    def extract_rates(**context) -> None:
        params = {"base": BASE_CURRENCY, "symbols": ",".join(TARGET_CURRENCIES)}
        response = requests.get(FRANKFURTER_URL, params=params, timeout=30)
        response.raise_for_status()
        payload = response.json()
        context["ti"].xcom_push(key=RAW_RATES_XCOM_KEY, value=payload)

    @task(task_id="transform_rates")
    def transform_rates(**context) -> None:
        ti = context["ti"]
        raw_payload = ti.xcom_pull(task_ids="extract_rates", key=RAW_RATES_XCOM_KEY)
        rate_date = raw_payload["date"]
        retrieved_at = datetime.utcnow().isoformat()
        transformed_records = []
        for target_currency, exchange_rate in raw_payload["rates"].items():
            transformed_records.append({
                "base_currency": raw_payload["base"],
                "target_currency": target_currency,
                "exchange_rate": exchange_rate,
                "rate_date": rate_date,
                "source": "Frankfurter API",
                "retrieved_at": retrieved_at,
            })
        ti.xcom_push(key=TRANSFORMED_RATES_XCOM_KEY, value=transformed_records)

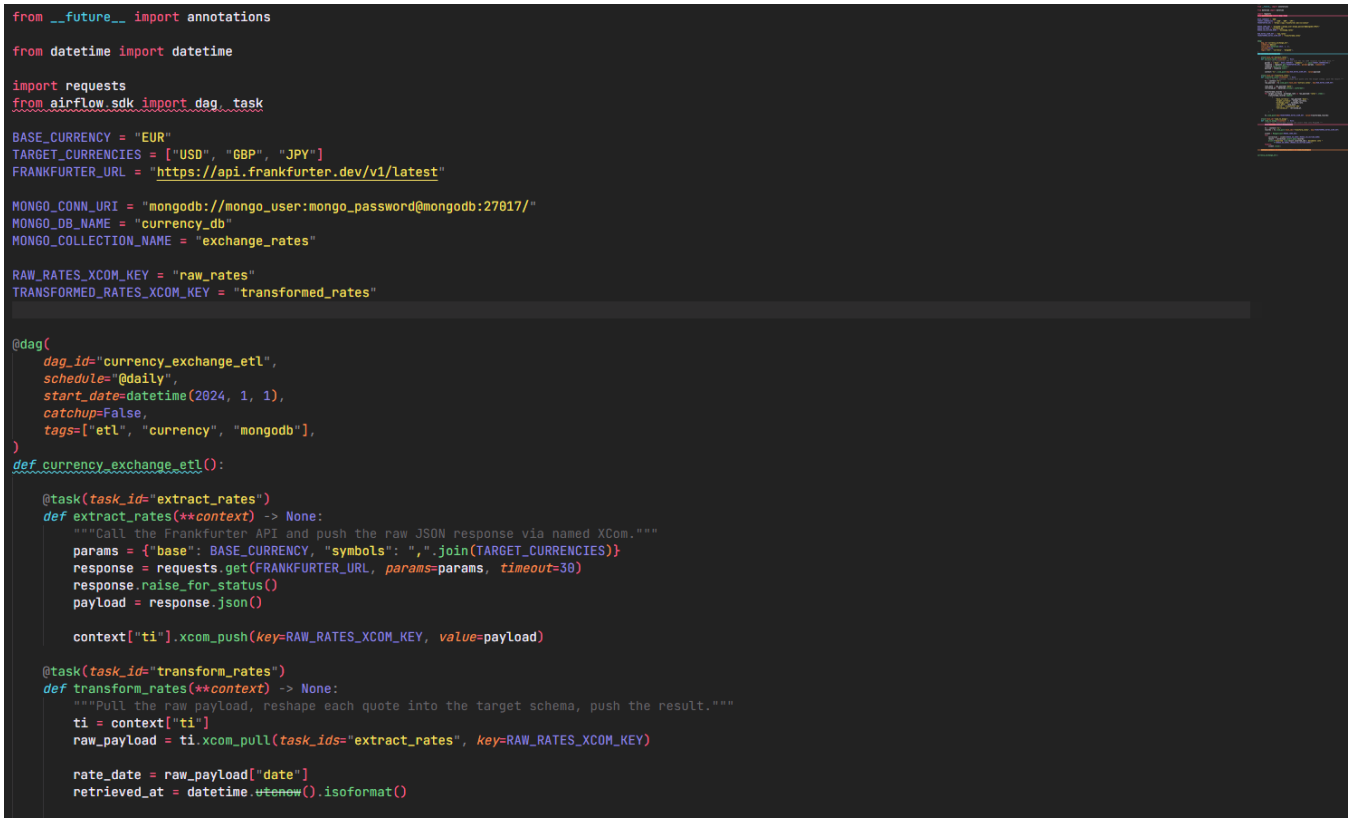
    @task(task_id="load_to_mongo")
    def load_to_mongo(**context) -> None:
        from pymongo import MongoClient
```

```
        ti = context["ti"]
        records = ti.xcom_pull(task_ids="transform_rates", key=TRANSFORMED_RATES_XCOM_KEY)
        client = MongoClient(MONGO_CONN_URI)
        try:
            collection = client[MONGO_DB_NAME][MONGO_COLLECTION_NAME]
            result = collection.insert_many(records)
            print(f"Inserted {len(result.inserted_ids)} documents into "
                  f"{MONGO_DB_NAME}.{MONGO_COLLECTION_NAME}")
        finally:
            client.close()

    extract_rates() >> transform_rates() >> load_to_mongo()


currency_exchange_etl()
```

4.1 Screenshot: DAG source code (editor)



```python
from __future__ import annotations

from datetime import datetime

import requests
from airflow.sdk import dag, task

BASE_CURRENCY = "EUR"
TARGET_CURRENCIES = ["USD", "GBP", "JPY"]
FRANKFURTER_URL = "https://api.frankfurter.dev/v1/latest"

MONGO_CONN_URI = "mongodb://mongo_user:mongo_password@mongodb:27017/"
MONGO_DB_NAME = "currency_db"
MONGO_COLLECTION_NAME = "exchange_rates"

RAW_RATES_XCOM_KEY = "raw_rates"
TRANSFORMED_RATES_XCOM_KEY = "transformed_rates"


@dag(
    dag_id="currency_exchange_etl",
    schedule="@daily",
    start_date=datetime(2024, 1, 1),
    catchup=False,
    tags=["etl", "currency", "mongodb"],
)
def currency_exchange_etl():

    @task(task_id="extract_rates")
    def extract_rates(**context) -> None:
        """Call the Frankfurter API and push the raw JSON response via named XCom."""
        params = {"base": BASE_CURRENCY, "symbols": ",".join(TARGET_CURRENCIES)}
        response = requests.get(FRANKFURTER_URL, params=params, timeout=30)
        response.raise_for_status()
        payload = response.json()

        context["ti"].xcom_push(key=RAW_RATES_XCOM_KEY, value=payload)

    @task(task_id="transform_rates")
    def transform_rates(**context) -> None:
        """Pull the raw payload, reshape each quote into the target schema, push the result."""
        ti = context["ti"]
        raw_payload = ti.xcom_pull(task_ids="extract_rates", key=RAW_RATES_XCOM_KEY)

        rate_date = raw_payload["date"]
        retrieved_at = datetime.utcnow().isoformat()
```

Figure 1 — currency_etl_dag.py opened in the editor, showing imports, configuration constants, and the extract_rates / transform_rates tasks.

5. Docker Services: MongoDB and Mongo Express

File: docker-compose.override.yml — merged automatically by Astro CLI into the same Docker Compose project as the Airflow services, so all containers share one network.

```
services:
  mongodb:
    image: mongo:7.0
    container_name: currency-etl-mongodb
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: mongo_user
      MONGO_INITDB_ROOT_PASSWORD: mongo_password
    ports:
      - "27017:27017"
    volumes:
      - mongodb_data:/data/db
    networks:
      - airflow

  mongo-express:
    image: mongo-express:1.0.2
    container_name: currency-etl-mongo-express
    restart: unless-stopped
    depends_on:
      - mongodb
    environment:
      ME_CONFIG_MONGODB_SERVER: mongodb
      ME_CONFIG_MONGODB_PORT: 27017
      ME_CONFIG_MONGODB_ADMINUSERNAME: mongo_user
      ME_CONFIG_MONGODB_ADMINPASSWORD: mongo_password
      ME_CONFIG_BASICAUTH_USERNAME: admin
      ME_CONFIG_BASICAUTH_PASSWORD: admin_password
    ports:
      - "8081:8081"
    networks:
      - airflow

volumes:
  mongodb_data:

networks:
  airflow:
    name: airflow-currency-etl_e837e7_airflow
    external: true
```

5.1 Screenshot: Running containers

```
        Dockerfile
        README.md
        dags/
        docker-compose.override.yml
        packages.txt
        requirements.txt

nothing added to commit but untracked files present (use "git add" to track)
PS D:\Task 3\airflow-currency-etl> docker ps
CONTAINER ID   IMAGE                                          COMMAND                  CREATED         STATUS         PORTS                                                  NAMES
7bc19b1d3c57   airflow-currency-etl_e837e7/airflow:latest   "tini -- /entrypoint…"   7 minutes ago   Up 7 minutes                                                          airflow-currency-etl_e837e
7-triggerer-1
e1177afaa0d5   airflow-currency-etl_e837e7/airflow:latest   "tini -- /entrypoint…"   7 minutes ago   Up 7 minutes   127.0.0.1:8080->8080/tcp                               airflow-currency-etl_e837e
7-api-server-1
e5565250e702   airflow-currency-etl_e837e7/airflow:latest   "tini -- /entrypoint…"   7 minutes ago   Up 7 minutes                                                          airflow-currency-etl_e837e
7-scheduler-1
4b7b4a42d37b   airflow-currency-etl_e837e7/airflow:latest   "tini -- /entrypoint…"   7 minutes ago   Up 7 minutes                                                          airflow-currency-etl_e837e
7-dag-processor-1
33a2261bcbb1   mongo-express:1.0.2                          "/sbin/tini -- /dock…"   7 minutes ago   Up 7 minutes   0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp            currency-etl-mongo-express
a61ecb6eacc9   postgres:15                                  "docker-entrypoint.s…"   7 minutes ago   Up 7 minutes   127.0.0.1:5432->5432/tcp                               airflow-currency-etl_e837e
7-postgres-1
ee1c1b202b1a   mongo:7.0                                    "docker-entrypoint.s…"   7 minutes ago   Up 7 minutes   0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp        currency-etl-mongodb
PS D:\Task 3\airflow-currency-etl>
```

Figure 2 — docker ps output showing all 7 containers running together: Airflow triggerer, api-server, scheduler, dag-processor, Postgres, MongoDB, and Mongo Express.

6. Pipeline Execution Results

6.1 Screenshot: DAG run — all tasks succeeded

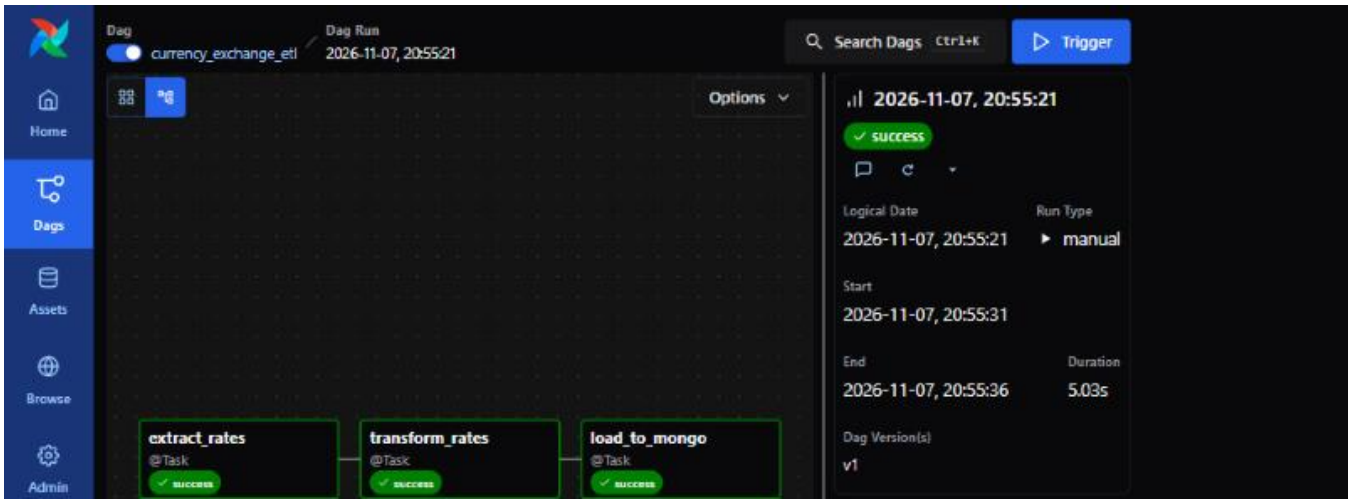


Figure 3 — Airflow UI graph view of currency_exchange_etl: extract_rates !' transform_rates !' load_to_mongo, all marked SUCCESS.

6.2 Screenshot: DAG graph structure

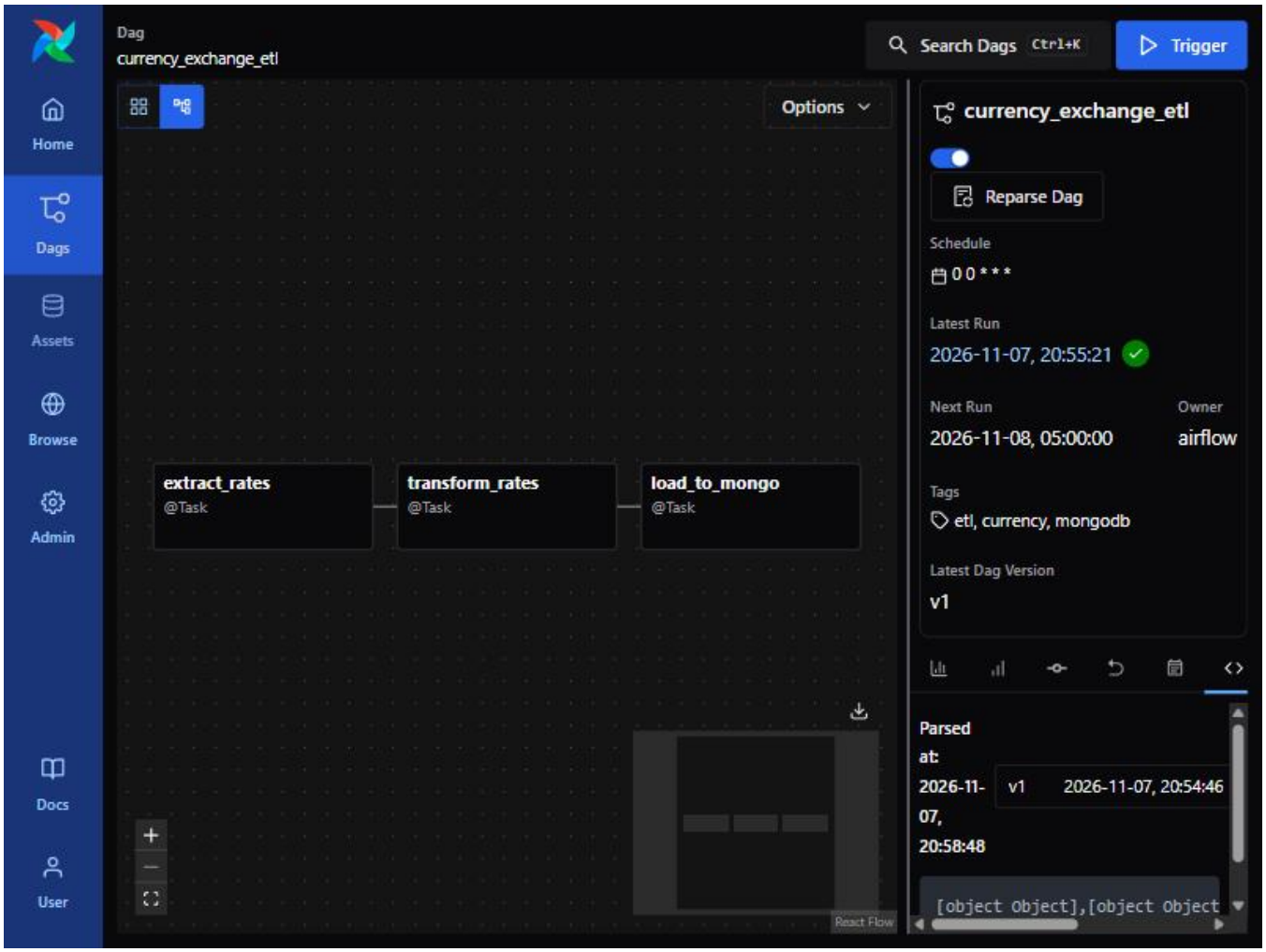


Figure 4 — DAG graph confirming the linear three-task dependency chain.

6.3 Screenshot: load_to_mongo task log



Figure 5 — Task log for load_to_mongo showing "Inserted 3 documents into currency_db.exchange_rates", confirming the write to MongoDB.

6.4 Screenshot: Data verified in Mongo Express

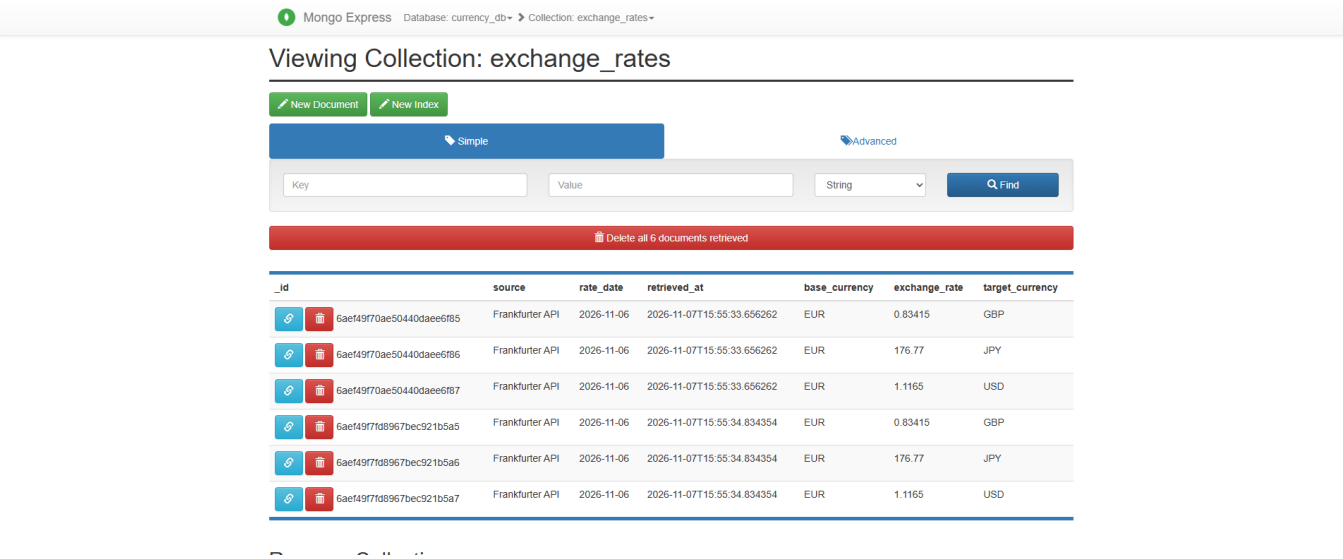


Figure 6 — Mongo Express UI showing the exchange_rates collection inside currency_db, with documents matching the required schema (base_currency, target_currency, exchange_rate, rate_date, source, retrieved_at).

7. Named XCom Usage

Per the assignment requirement, tasks communicate exclusively through named Airflow XComs rather than return-value XComs:

extract_rates pushes the raw API response under key "raw_rates".

transform_rates pulls "raw_rates" from extract_rates, and pushes the reshaped records under key "transformed_rates".

load_to_mongo pulls "transformed_rates" from transform_rates and writes them to MongoDB.

8. Source Code Repository

The complete project (DAG, Dockerfile, Docker Compose service definitions, and this report) has been pushed to GitHub as a sequence of incremental commits, one per subtask:

https://github.com/Munam32/airflow-currency-etl

9. Conclusion

The ETL pipeline was implemented and verified end-to-end: daily EUR exchange rates are extracted from the Frankfurter API, transformed into the required normalized schema, and loaded into MongoDB, with MongoDB and Mongo Express running as properly networked, persistent Docker services alongside Airflow, and with all inter-task data passed through named XComs as required.